

Módulo 9 – Capítulo 10 – Sección 01

Security by design en sistemas de IA: integrar seguridad desde la arquitectura

Los nueve capítulos anteriores de este módulo presentaron los controles de seguridad de los sistemas de IA capa por capa: threat modeling, prompt injection, ataques adversariales, red teaming, RAG poisoning, privacidad de datos, hardening del endpoint, trazabilidad y auditoría, y compliance regulatorio. Este capítulo de síntesis presenta los principios arquitectónicos que unifican todos esos controles en un sistema coherente: security by design, mínimo privilegio, defense in depth, threat modeling continuo, y security testing pipeline. Sin estos principios de diseño, los controles individuales son decisiones aisladas que cada ingeniero aplica o no según su iniciativa; con ellos, la seguridad es una propiedad del sistema que emerge del diseño deliberado en cada decisión arquitectónica.

Security by design es el principio que sostiene que la seguridad es más efectiva y más barata cuando se incorpora desde la fase de diseño arquitectónico que cuando se añade como capa posterior a un sistema ya construido. Las razones son tanto técnicas como económicas. Técnicamente, las decisiones arquitectónicas tempranas determinan el conjunto posible de controles de seguridad disponibles: un sistema diseñado sin separación de privilegios entre el plano de datos y el plano de acción no puede añadirla fácilmente en producción porque habría que refactorizar la interfaz entre el LLM y el sistema de herramientas; un sistema que almacena el historial de conversación sin cifrado por usuario no puede implementar el derecho al olvido del GDPR sin rediseñar el esquema de almacenamiento. Económicamente, un defecto de diseño de seguridad encontrado en la fase de diseño cuesta unidades de esfuerzo de ingeniería; el mismo defecto encontrado en producción después de un incidente cuesta órdenes de magnitud más en análisis forense, remediation, comunicación de crisis y daño reputacional.

Security by design en sistemas de IA opera en tres niveles que se refuerzan mutuamente. El nivel de arquitectura del sistema determina cómo los componentes se conectan: la elección de una API de proveedor externo versus un modelo self-hosted determina el threat model de confidencialidad de los datos; la elección de un vectorstore compartido versus uno por-tenant

determina el riesgo de cross-tenant contamination en RAG; la elección de un bus de mensajes versus llamadas directas entre componentes determina la superficie de ataque de la pipeline agéntica. El nivel de diseño de datos determina qué datos se recolectan, cómo se almacenan, y quién puede acceder: la minimización de datos desde el diseño significa recolectar solo lo necesario, no recolectar todo lo disponible y filtrar después. El nivel de diseño de la interfaz determina cómo los inputs del usuario llegan al modelo y cómo los outputs llegan al usuario: el diseño del sistema de prompts con separadores estructurales, los límites de longitud de input en tokens, y los clasificadores en el output son decisiones de interfaz con implicaciones de seguridad directas.

Los secure defaults son el mecanismo operacional que implementa security by design en el día a día del equipo de ingeniería. Todos los componentes del sistema deben configurarse de forma segura por defecto —cifrado habilitado en el vectorstore, autenticación requerida en todos los endpoints, logging activo, rate limiting configurado— y la configuración insegura debe requerir un override explícito con justificación documentada. Esto aplica al serving layer (vLLM y TGI que no habilitan TLS por defecto deben ser configurados explícitamente con TLS antes del primer deployment a un ambiente no local), al vectorstore (Weaviate y pgvector que no cifran en reposo por defecto requieren configuración explícita), y al pipeline de orquestación (LangGraph y LangChain que no implementan privilege separation por defecto requieren diseño explícito del plano de control).

Aspectos técnicos

- **Decisiones de arquitectura con implicaciones de seguridad:** API de proveedor vs. self-hosted (confidencialidad), vectorstore compartido vs. por-tenant (cross-tenant contamination), bus de mensajes vs. llamadas directas (superficie de ataque de pipeline agéntica), arquitectura de microservicios vs. monolítica (blast radius de un compromiso).
- **Trust zones y boundaries explícitas:** el diseño debe definir qué código y datos son confiables (system prompt del operador), cuáles son semi-confiables (historial de conversación del usuario), y cuáles son untrusted (documentos externos, outputs de sub-agentes, web pages recuperadas por el agente).
- **Secure defaults:** TLS en todos los endpoints (incluidos los internos), autenticación requerida antes de cualquier operación, logging activo por defecto, rate limiting configurado desde el deployment inicial; la configuración insegura requiere override explícito documentado.
- **Fail securely:** si el clasificador de seguridad no está disponible, la request debe fallar cerrada (bloqueada) no abierta (permitida sin clasificación); si el plano de control no

puede verificar la autorización de una tool call, la herramienta no se ejecuta; el sistema de IA debe definir explícitamente su comportamiento ante fallos de sus controles de seguridad.

- **Security architecture review:** revisión formal del diseño por un equipo de seguridad antes del primer deployment, con findings documentados y mitigaciones verificadas; los findings críticos deben resolverse antes del go-live, no planificarse para "la próxima iteración".

Nota del Arquitecto: La pregunta que más revela si un equipo practica security by design es "¿cuándo habló el equipo de seguridad por primera vez sobre este sistema?". Si la respuesta es "antes de escribir código", el equipo practica security by design. Si la respuesta es "cuando el sistema estaba en staging y alguien preguntó qué controles de seguridad había", el equipo añade seguridad post-hoc. La diferencia en el costo total de ownership a 12 meses es notable: el primer equipo invierte tiempo en arquitectura; el segundo invierte ese mismo tiempo más mucho más en remediación.

La seguridad añadida post-hoc a un sistema de IA en producción siempre es más costosa, menos efectiva, y con más brechas que la seguridad diseñada desde el inicio: las decisiones de trust zones, sandboxing, separación de privilegios y cifrado deben tomarse antes de la primera línea de código.

Módulo 9 – Capítulo 10 – Sección 02

Principio de mínimo privilegio aplicado a modelos, agentes y herramientas

¿Cuánta información necesita realmente el modelo para responder la pregunta actual del usuario? ¿Cuántas herramientas necesita el agente para completar la tarea que se le asignó? ¿Qué permisos de sistema de archivos necesita el proceso de ejecución de código? Las respuestas a estas preguntas, cuando se aplican de manera sistemática en el diseño del sistema, definen la implementación del principio de mínimo privilegio (Principle of Least Privilege, PoLP) en sistemas de IA. El PoLP, formulado por Jerome Saltzer y Michael Schroeder en 1975 en el contexto de sistemas operativos, establece que cada componente debe tener acceso únicamente a los recursos estrictamente necesarios para su función, y ni uno más. En sistemas de IA, este principio tiene implicaciones concretas y operacionalmente significativas en al menos tres dimensiones: el contexto del modelo, el acceso a herramientas del agente, y los permisos del entorno de ejecución.

En la dimensión del contexto del modelo, el PoLP significa que el LLM solo debe recibir en su ventana de contexto los datos estrictamente necesarios para la tarea actual. Un sistema RAG que incluye los 20 chunks más similares a la query cuando 3 o 4 serían suficientes viola el principio: el modelo recibe información innecesaria que amplía la superficie de exposición de datos sensibles y aumenta el riesgo de que instrucciones adversariales en documentos menos relevantes sean procesadas por el modelo. Un sistema de conversación multi-turn que incluye todo el historial de conversación desde el inicio de la sesión cuando solo los últimos 10 turnos son relevantes viola el principio: los datos de conversación temprana que el usuario mencionó (incluyendo posible PII) permanecen en el contexto sin necesidad funcional. Un sistema agéntico que incluye en el contexto del modelo las credenciales de conexión a bases de datos o los tokens de acceso a APIs externas viola el principio de la manera más crítica: esas credenciales son extraíbles mediante prompt leaking y no tienen ningún motivo para estar en el contexto del modelo.

En la dimensión del acceso a herramientas, la diferencia entre un agente seguro y uno vulnerable con frecuencia es la diferencia entre herramientas con scope específico y

herramientas con acceso general. Un agente que necesita leer el estado de los pedidos de un cliente no debe tener acceso a una herramienta de "base de datos completa"; debe tener acceso a una función `get_order_status(customer_id, order_id)` que solo puede ejecutar SELECT sobre la tabla de pedidos y solo para el `customer_id` que corresponde al usuario autenticado de la sesión actual. La diferencia en el impacto de un compromiso es radicalmente diferente: un agente con acceso a la base de datos completa que es comprometido tiene acceso a todos los datos de todos los clientes; un agente con acceso solo a `get_order_status` que es comprometido puede leer el estado de pedidos del cliente actual y nada más.

La dimensión del entorno de ejecución aplica el mismo principio al proceso que ejecuta el código del agente. El proceso no debe tener permisos de escritura en directorios que solo necesita leer; no debe tener acceso de red a segmentos que no necesita contactar; no debe tener credenciales de sistema que no necesita usar. En entornos cloud, el IAM role del agente debe ser el más restrictivo posible: acceso solo a los servicios y recursos específicos que el agente necesita invocar, sin wildcards en recursos ni en acciones, con revisión periódica para reducir permisos cuando el scope de la tarea del agente cambie.

Aspectos técnicos

- **Mínimo privilegio en el contexto del modelo:** incluir solo los documentos RAG más relevantes (top-k conservador, 3-5 chunks), solo el historial de conversación reciente (últimas N turns), y ningún dato que el modelo no necesita para la tarea actual; credenciales y tokens nunca en el contexto del LLM.
- **Tool scoping para agentes:** herramientas especializadas con scope por operación y recurso específico (`get_customer_info(customer_id)`) en lugar de herramientas genéricas con acceso amplio; el agente no recibe ni puede inferir las credenciales de conexión subyacentes.
- **Filesystem access mínimo:** montar solo el directorio específico necesario con los permisos estrictamente requeridos (solo lectura si la tarea no requiere escritura); usar chroot, namespace isolation, o volúmenes de Docker con scope limitado para garantizar que el agente no puede acceder a paths fuera de su scope aunque sea comprometido.
- **IAM least privilege en cloud:** el IAM role del agente en AWS, Azure o GCP especifica permisos en servicios y recursos específicos sin wildcards (`s3:GetObject` en el bucket específico, no `s3:*` en todos los buckets); revisión trimestral de permisos para reducir scope cuando las tareas del agente evolucionen.

- **Temporal privilege elevation:** para operaciones que requieren permisos más altos (acciones irreversibles, modificación de configuración), implementar elevación temporal de privilegio con tiempo de vida corto (TTL de minutos) y logging explícito de cada elevación, en lugar de mantener privilegios elevados de forma permanente.

Nota del Arquitecto: *El principio de mínimo privilegio tiene un costo operacional real: diseñar herramientas con scope específico requiere más trabajo que exponer una interfaz de base de datos genérica. Pero ese costo es una inversión que se recupera en el momento en que el agente es comprometido — que en sistemas en producción con miles de usuarios, es una cuestión de cuándo, no de si. Un agente con mínimo privilegio comprometido es un incidente contenido; un agente sin mínimo privilegio comprometido es una brecha de seguridad completa que afecta a todos los usuarios y sistemas a los que el agente tenía acceso.*

El mínimo privilegio en sistemas de IA es la diferencia entre un compromiso contenido —el agente solo pudo acceder a lo que necesitaba, el daño está limitado a esa porción del sistema— y un compromiso catastrófico donde el agente comprometido tenía acceso a toda la infraestructura. Implementarlo requiere trabajo adicional en el diseño de cada herramienta y del entorno de ejecución, pero ese trabajo es la inversión de ingeniería con mayor retorno en términos de reducción del blast radius de cualquier ataque exitoso.

Módulo 9 – Capítulo 10 – Sección 03

Defense in depth: capas de control a nivel de prompt, API, red y datos

Si hubiera un solo control de seguridad que funcionara con perfecta efectividad contra todos los ataques conocidos y futuros a sistemas de IA, el diseño de seguridad sería simple: implementar ese control y el trabajo estaría hecho. La realidad es la contraria: cada control tiene sus límites de efectividad. LlamaGuard clasifica con alta precisión pero tiene falsos negativos ante perturbaciones adversariales bien diseñadas. Las instrucciones de refuerzo en el system prompt elevan el costo del jailbreak pero no eliminan su posibilidad. El rate limiting contiene los ataques de extracción masiva pero no los distribuidos con timing cuidadoso. Ante esta realidad, la arquitectura correcta no busca el control perfecto sino la combinación de múltiples controles independientes donde el fallo de cualquiera de ellos no compromete el sistema completo. Este es el principio de defense in depth: cada capa reduce de forma independiente la probabilidad de un ataque exitoso de extremo a extremo.

Defense in depth en sistemas de IA se implementa en cuatro capas que operan en niveles de abstracción diferentes y tienen superficies de ataque distintas. Esta independencia es la condición que hace que la arquitectura sea robusta: si el clasificador de output y el sistema de logging usaran la misma API key, un atacante que compromete esa key deshabilita dos controles simultáneamente; si comparten dependencias de red, un ataque de disponibilidad a esa red deshabilita ambas capas a la vez. Las cuatro capas deben diseñarse para tener diferentes superficies de ataque, diferentes dependencias, y diferentes mecanismos de fallo para que sean genuinamente independientes.

La primera capa opera a nivel del prompt: el system prompt con instrucciones de seguridad explícitas, los separadores estructurales que distinguen el contexto del sistema, los documentos recuperados por RAG y el input del usuario, y las instrucciones de refuerzo que instruyen al modelo a no seguir instrucciones encontradas en documentos externos. Esta capa es la más cercana al comportamiento del modelo y actúa directamente sobre su interpretación del contexto. Es también la más vulnerable a un adversario sofisticado, razón por la cual nunca debe ser la única capa.

La segunda capa opera a nivel de la API: input validation con LlamaGuard y Prompt Guard antes de que el input llegue al modelo, output validation con LlamaGuard y Presidio después del output del modelo y antes de que llegue al usuario, rate limiting multidimensional, autenticación con tokens JWT granulares, y logging de cada request con safety scores. Esta capa actúa como el sistema de verificación activa del comportamiento del sistema: independientemente de lo que el modelo decida hacer, la API valida que lo que entra y sale cumple las políticas del sistema.

La tercera capa opera a nivel de red: WAF tanto en la capa HTTP convencional (AWS WAF, Cloudflare) como en la capa semántica especializada para IA (Lakera Guard, Prompt Security), TLS 1.3 con verificación de certificados en todos los saltos del pipeline, segmentación de red que aísla el serving layer del LLM de otros componentes del sistema (el servidor vLLM no debe tener conectividad directa a la base de datos de usuarios), y network egress filtering para agentes con acceso web que previene conexiones a IPs privadas y dominios no autorizados.

La cuarta capa opera a nivel de datos: cifrado AES-256 del vectorstore y logs de inferencia con claves KMS rotadas, controles de acceso RBAC a los índices vectoriales donde un usuario solo puede recuperar documentos de su tenant, inmutabilidad de logs con S3 Object Lock o firma criptográfica HMAC-SHA256, y auditoría de acceso a datos sensibles. Esta capa garantiza que incluso si un atacante logra eludir las tres capas anteriores, los datos que accede están protegidos por controles independientes.

Aspectos técnicos

- **Capa 1 - Prompt:** system prompt con instrucciones explícitas de seguridad y tags estructurales (`<system>` , `<retrieved_context>` , `<user_message>`), instrucciones de no revelar el system prompt, instrucciones de no seguir instrucciones en documentos recuperados; primera barrera de comportamiento del modelo.
- **Capa 2 - API:** LlamaGuard/Prompt Guard en input, LlamaGuard/Presidio en output, rate limiting multidimensional en Redis, autenticación JWT con claims granulares, logging de safety scores por request; verificación activa independiente del comportamiento del modelo.
- **Capa 3 - Red:** AWS WAF + Lakera Guard en paralelo, TLS 1.3 con certificate pinning, segmentación de red entre serving layer y otros componentes, network egress filtering con allowlist de dominios para agentes; protección perimetral y semántica complementarias.

- **Capa 4 - Datos:** AES-256 para vectorstore y logs con claves KMS, RBAC en índices vectoriales por tenant, S3 Object Lock COMPLIANCE para logs inmutables, auditoría de acceso a PHI y PII; protección que opera incluso si las capas anteriores son eludidas.
- **Testing de independencia:** en el red teaming, incluir escenarios donde una capa está deshabilitada y verificar que las capas restantes contienen el daño; la interdependencia entre capas (una asumiendo que la anterior ya validó) es una fuente de vulnerabilidades sistémicas.

Nota del Arquitecto: *Un error frecuente en la implementación de defense in depth es crear capas que son conceptualmente independientes pero técnicamente dependientes: el clasificador de input y el clasificador de output usan el mismo modelo, por lo que un adversario que aprende a evadir uno ha aprendido a evadir el otro. La independencia real requiere diversidad técnica: clasificadores con arquitecturas diferentes, frameworks de validación distintos, mecanismos de detección complementarios. La diversidad tiene un costo de mantenimiento, pero produce la independencia de fallo que el principio requiere.*

Un atacante que logra eludir la capa de prompt de un sistema con defense in depth correctamente implementada aún debe eludir el clasificador de output (segunda capa), el WAF semántico (tercera capa), y el control de acceso a datos (cuarta capa) para lograr un impacto significativo. Cada capa reduce independientemente la probabilidad de un ataque de extremo a extremo, y la probabilidad combinada de eludir todas las capas es el producto de las probabilidades individuales — un número significativamente más pequeño que cualquiera por separado.

Módulo 9 – Capítulo 10 – Sección 04

Threat modeling continuo: actualizar el modelo de amenazas con cada nueva capacidad

El threat model de un sistema de IA no es un documento que se escribe una vez durante el diseño inicial, se revisa en la primera reunión de seguridad, y luego vive archivado en Confluence hasta que alguien lo menciona en una retrospectiva. Es un artefacto vivo que debe evolucionar en paralelo con las capacidades del sistema, porque las capacidades de un sistema de IA determinan directamente su superficie de ataque: cada nueva capacidad abre nuevas posibilidades de uso y, con ellas, nuevos vectores de ataque que no existían antes. Tratar el threat model como un documento estático es equivalente a desarrollar una aplicación web, hacer el análisis de seguridad inicial, y nunca revisarlo cuando se añaden nuevas rutas, se cambia la base de datos, o se integran APIs externas.

La razón por la que los sistemas de IA hacen especialmente crítica esta actualización continua es la velocidad con la que sus capacidades cambian. En sistemas de software tradicional, añadir una nueva funcionalidad generalmente expande la superficie de ataque de forma acotada y predecible: una nueva ruta de API tiene sus propias validaciones de input y control de acceso. En un sistema de IA, cambiar el modelo base de Claude 3.5 Sonnet a Claude 3.7 puede introducir capacidades de razonamiento extendido (extended thinking) que crean nuevos comportamientos ante ataques multi-turn; añadir visión multimodal introduce el vector de prompt injection a través de imágenes (instrucciones adversariales en imágenes que el modelo lee pero el humano no detecta); añadir memoria persistente entre sesiones crea el riesgo de que datos de una sesión contaminen decisiones en sesiones futuras. Cada uno de estos cambios invalida parte del threat model anterior e introduce nuevas amenazas que deben ser analizadas explícitamente.

El proceso de actualización del threat model se articula en seis pasos que deben ejecutarse cada vez que ocurre un trigger de cambio significativo. Primero, documentar la nueva capacidad con precisión técnica: no "añadimos búsqueda web" sino "el agente ahora puede usar la herramienta `web_search(query)` que hace requests a un proxy egress y devuelve el texto de las páginas, incluyendo metadatos y comentarios HTML". Esta precisión determina

qué amenazas son relevantes. Segundo, actualizar el Data Flow Diagram para incluir los nuevos flujos de datos y componentes. Tercero, aplicar STRIDE a cada nuevo flujo y componente. Cuarto, consultar MITRE ATLAS para las TTPs (Tactics, Techniques, and Procedures) relevantes a la nueva capacidad. Quinto, estimar el riesgo de cada nueva amenaza con la combinación de likelihood e impact. Sexto, definir los controles de mitigación para los riesgos que superen el umbral de aceptación, con owner y fecha de implementación.

Mantener el threat model en el repositorio del proyecto como código —en YAML o JSON usando notaciones como PASTA o STRIDE-in-YAML, con herramientas como Threatspec u OWASP Threat Dragon— tiene varias ventajas operacionales sobre el documento de texto tradicional. La historia de cambios en git muestra exactamente cuándo se añadió cada amenaza y con qué motivación; el diff entre versiones permite verificar que cada cambio de capacidad fue acompañado de la actualización correspondiente del threat model; las discrepancias entre la historia de git del código y la historia del threat model revelan cambios de capacidad que no fueron analizados desde la perspectiva de seguridad.

Aspectos técnicos

- **Triggers de actualización obligatoria:** cambio de modelo base (de Claude 3.5 a una nueva versión), adición o modificación de herramientas en el agente, expansión del corpus RAG con fuentes externas no controladas, habilitación de multimodalidad (vision, audio, code execution), adición de memoria persistente entre sesiones, cambio en el perfil de usuarios (de internos a públicos).
- **Proceso de actualización:** documentar la nueva capacidad con precisión técnica → actualizar el DFD → aplicar STRIDE a los nuevos flujos → consultar MITRE ATLAS para TTPs relevantes → estimar likelihood × impact de cada amenaza → definir controles de mitigación con owner y fecha → registrar en changelog del threat model.
- **Threat model como código:** formato YAML/JSON en el repositorio del proyecto con historia de cambios en git; herramientas como Threatspec, OWASP Threat Dragon, o IriusRisk para gestión y generación de informes; el diff entre versiones es la evidencia de que el threat model se actualizó ante cada cambio de capacidad.
- **Threat intelligence para IA:** suscribirse a fuentes específicas de amenazas para sistemas de IA: MITRE ATLAS updates, OWASP LLM Top 10 actualizaciones, publicaciones de Anthropic/OpenAI/Google sobre vulnerabilidades descubiertas, Hugging Face Security Advisories; incorporar nuevas técnicas de ataque al threat model antes de que sean explotadas en producción.

- **Validación mediante red teaming:** cada actualización del threat model debe trigger un ejercicio de red teaming enfocado en las nuevas amenazas identificadas; el threat model sin red teaming de validación es teoría sin verificación empírica, y la teoría sin verificación puede crear una falsa sensación de seguridad.

Nota del Arquitecto: *El threat model tiene un valor secundario que frecuentemente se subestima: como herramienta de comunicación. Cuando el equipo de producto propone añadir una nueva funcionalidad al agente —por ejemplo, capacidad de enviar emails en nombre del usuario— el threat model actualizado es el mecanismo que convierte esa discusión de "sí/no" a "sí, con estos controles específicos". El threat model da al equipo de ingeniería el vocabulario y el artefacto para participar en decisiones de producto con argumentos de seguridad concretos y cuantificables, no solo con intuiciones.*

El threat model debe revisarse obligatoriamente antes de cualquier release significativo del sistema de IA y periódicamente en producción para incorporar nuevas técnicas de ataque documentadas en la comunidad de seguridad. El landscape de amenazas para sistemas de IA evoluciona significativamente más rápido que el de sistemas de software tradicional: lo que no era un vector de ataque conocido hace seis meses puede ser una técnica documentada y ampliamente replicada hoy.

Módulo 9 – Capítulo 10 – Sección 05

Security testing pipeline y matriz de control: verificación continua y referencia operacional

Los principios de security by design, mínimo privilegio y defense in depth que articularon las secciones anteriores de este capítulo son necesarios pero insuficientes sin un mecanismo que verifique continuamente que los controles implementados funcionan como se diseñaron. Los controles de seguridad degradan con el tiempo: las reglas de un WAF dejan de detectar nuevas variantes de ataque; los clasificadores de seguridad como LlamaGuard experimentan drift ante técnicas de evasión que evolucionan; las configuraciones de seguridad se modifican inadvertidamente durante deployments; las dependencias de seguridad acumulan vulnerabilidades. El security testing pipeline es la respuesta de ingeniería a esta degradación: un conjunto de verificaciones automatizadas que corren continuamente en el ciclo de desarrollo y deployment para detectar regresiones de seguridad antes de que lleguen a producción o, si ya están en producción, antes de que sean explotadas.

El security testing pipeline para sistemas de IA combina técnicas del security testing de software tradicional con técnicas específicas de IA. SAST (Static Application Security Testing) con Semgrep y Bandit detecta vulnerabilidades en el código de la aplicación que rodea al modelo: concatenación directa de user input en prompts sin separadores estructurales, uso de `pickle.load` para deserializar artefactos de modelo (vulnerable a deserialization attacks), API keys hardcodeadas en código o en archivos de configuración versionados, y logging de inputs completos del usuario sin cifrado. Las reglas SAST para proyectos de IA deben incluir estas verificaciones específicas que no existen en los rulesets genéricos.

DAST (Dynamic Application Security Testing) con OWASP ZAP o Burp Suite prueba el endpoint de la API de IA en ejecución contra ataques de capa HTTP: verificar que todos los endpoints requieren autenticación (no solo los endpoints de inferencia sino también los de administración, health check, y métricas), verificar que el rate limiting devuelve HTTP 429 con Retry-After correcto, verificar que los headers de seguridad están presentes (Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security), y verificar que los

mensajes de error no exponen información sensible del sistema (versiones de librerías, rutas internas, mensajes de stack trace).

El fuzzing de prompts automatiza la generación y ejecución de variaciones adversariales sobre el sistema: toma prompts de ataque conocidos como baseline y genera mutaciones mediante inserción de caracteres especiales, sustitución por homógrafos, traducción parcial a otros idiomas, fragmentación en múltiples messages, y codificación en Base64 o ROT13. Herramientas como Garak (NVIDIA) y un subset de PyRIT (Microsoft) pueden ejecutarse en el pipeline de CI/CD como gates de deployment: si la tasa de activación de safety filters cae por debajo de un umbral esperado ante categorías de ataque conocidas, el deployment se bloquea hasta que el equipo de seguridad revise la regresión.

La evaluación periódica de los clasificadores de seguridad cierra el ciclo: LlamaGuard, Azure Content Safety y otros clasificadores usados como controles deben evaluarse semanalmente con un dataset de golden tests que incluya tanto true positives (contenido dañino que debe ser bloqueado) como false positives (contenido benigno que no debe bloquearse). La degradación del F1 score del clasificador en el tiempo puede indicar model drift o nuevas técnicas de evasión que el clasificador no conoce, y debe trigger una revisión antes de que el gap entre el clasificador y las técnicas actuales de ataque sea explotado.

Matriz de control del Módulo 9

La tabla siguiente mapea las amenazas principales del módulo con sus controles de mitigación, el capítulo donde se desarrolla el control, y las herramientas de implementación. Esta matriz es el artefacto de referencia operacional que consolida los controles de los diez capítulos en formato consultable.

Amenaza	Control técnico	Capítulo	Herramienta de implementación
Prompt injection directa	Separadores estructurales + instrucciones de refuerzo + clasificador de input	Cap. 02	Prompt Guard (Meta), diseño de system prompt
Prompt injection indirecta (RAG)	Tratar documentos RAG como untrusted + validación de output antes de ejecutar acciones	Cap. 02, 05	Tags <code><retrieved_context></code> , output filter
Jailbreaking	Validación semántica del output independiente del modelo + defense in depth	Cap. 02	LlamaGuard-3 (Meta)
RAG poisoning	Validación de contenido en ingestión + RBAC al vectorstore + auditoría periódica del corpus	Cap. 05	LlamaGuard, Microsoft Presidio, pgvector RBAC
Cascading compromise multi-agente	Tratar output de sub-agentes como input untrusted + sandboxing de comunicaciones	Cap. 05	Validación de schema inter-agente, LlamaGuard
Ataques adversariales en guardrails	Múltiples clasificadores con arquitecturas distintas + ensemble de detección	Cap. 03	LlamaGuard + Azure Content Safety + Perspective API
Model extraction	Rate limiting multidimensional + monitoreo de distribución de topics	Cap. 07	Redis sliding window, análisis de embeddings
Memorización de training data	Deduplicación del corpus + DP-SGD durante fine-tuning	Cap. 06	Opacus (Meta), MinHash LSH
Data poisoning	Auditoría estadística del dataset + activation clustering + escaneo safetensors	Cap. 03, 01	modelscan, trivy, activación clustering
Supply chain attacks	Verificación de integridad de pesos (SHA256) + safetensors en lugar de pickle + SBOM	Cap. 01	modelscan, pip-audit, trivy
PII exposure	Detección automática de PII en input/output +	Cap. 06	Microsoft Presidio, Amazon Comprehend

Amenaza	Control técnico	Capítulo	Herramienta de implementación
	pseudonimización + cifrado por usuario		
Prompt leaking	No almacenar credenciales en system prompt + output filter para texto del system prompt	Cap. 02	Output filter con n-gram matching
Backdoors en modelos	Pruebas de comportamiento pre-deployment + verificación de integridad de artefactos	Cap. 03	Activation clustering, behavioral testing
Falta de trazabilidad	Audit log con schema completo + inmutabilidad + integración SIEM	Cap. 08	S3 Object Lock, Splunk/Sentinel, HMAC-SHA256
Incumplimiento GDPR	Borrado selectivo en logs + machine unlearning + minimización de datos	Cap. 09	Cifrado por usuario con TTL, SISA training
Incumplimiento EU AI Act	Risk management system + technical documentation + human oversight	Cap. 09	Control matrix, red team reports

Aspectos técnicos

- **SAST adaptado a IA:** reglas Sengrep personalizadas para detectar concatenación directa de user input en prompts, uso de pickle.load para artefactos de modelo, API keys en código, y logging sin cifrado de inputs del usuario; Bandit para vulnerabilidades Python en el código de la aplicación.
- **DAST para endpoints de IA:** OWASP ZAP con extensiones específicas para IA; verificar autenticación en todos los endpoints, rate limiting activo, headers de seguridad, y ausencia de información sensible en mensajes de error.
- **Fuzzing de prompts en CI/CD:** Garak como gate de deployment con subset de probes para categorías de ataque conocidas; umbral de activación de safety filter definido como SLA de seguridad que bloquea el deploy si no se cumple.
- **Evaluación de clasificadores:** dataset de golden tests (true positives + false positives) evaluado semanalmente; alerta si F1 score cae más del 5% respecto al baseline; triggerar revisión del clasificador o reentrenamiento si la degradación supera el umbral.

- **Pipeline completo en CI/CD:** SAST en cada PR (fallo si hallazgos críticos) → DAST en staging antes de cada deploy → evaluación de clasificadores semanal → red teaming automatizado (Garak) como gate de deployment → red teaming manual periódico para vulnerabilidades que requieren creatividad adversarial.

Nota del Arquitecto: La matriz de control tiene valor operacional solo si está actualizada. Un control marcado como "implementado" que fue deshabilitado hace tres meses por un incidente de falsos positivos pero no fue eliminado de la matriz es peor que no tener la matriz: da una falsa sensación de cobertura. Mantener la matriz como código en el repositorio del proyecto con CI que verifique automáticamente los controles que pueden verificarse de forma programática (cifrado habilitado, logs fluyendo al SIEM, clasificador respondiendo) es la forma de convertirla en un artefacto de referencia confiable en lugar de un documento decorativo.

El security testing pipeline automatizado proporciona la cobertura consistente que ningún proceso manual puede garantizar, y la matriz de control proporciona la referencia operacional que permite al equipo responder rápidamente a cualquier pregunta sobre el estado de cobertura de seguridad del sistema: qué amenazas están mitigadas, con qué controles, y dónde están los gaps.

Módulo 9 – Capítulo 10 – Sección 06

Cierre: un sistema de IA seguro no es el que nunca es atacado, sino el que sobrevive al ataque

El objetivo de AI Security Engineering no es construir un sistema impenetrable. Esa meta no existe en el dominio de los sistemas complejos reales: los sistemas de IA tienen superficies de ataque que evolucionan con cada nueva capacidad, los adversarios tienen recursos y creatividad que ningún equipo puede anticipar completamente, y la investigación en seguridad de IA produce nuevas técnicas de ataque con una velocidad que supera a la de los controles defensivos. El objetivo real es diferente y alcanzable: construir un sistema suficientemente costoso de atacar para disuadir a la mayoría de los adversarios, suficientemente resiliente para continuar operando cuando es atacado, suficientemente observable para detectar y contener los ataques que ocurren, y suficientemente adaptable para aprender de esos ataques y mejorar.

Este capítulo sintetizó los cinco principios arquitectónicos que hacen que un sistema de IA sea ese tipo de sistema. Security by design —la premisa de que las decisiones de seguridad más importantes se toman antes de escribir código, en la arquitectura del sistema— garantiza que los controles son coherentes con el diseño en lugar de ser parches añadidos sobre decisiones que los hacen inefectivos. Mínimo privilegio —dar a cada componente acceso solo a lo estrictamente necesario— limita el blast radius de cualquier compromiso a la porción mínima del sistema a la que ese componente tenía acceso. Defense in depth —múltiples capas de control independientes— garantiza que el fallo de cualquier control individual no compromete el sistema completo, porque quedan otras capas que contienen el daño. Threat modeling continuo —actualizar el modelo de amenazas con cada nueva capacidad— asegura que el análisis de seguridad evoluciona a la misma velocidad que el sistema. Y el security testing pipeline —verificación automatizada de que los controles funcionan— previene la degradación silenciosa de los controles con el tiempo.

Estos cinco principios no son nuevos: security by design, mínimo privilegio y defense in depth son principios fundamentales de la seguridad informática que datan de décadas. Lo que es nuevo es su aplicación a un tipo de sistema donde el componente central es un modelo de

lenguaje que genera outputs probabilísticos ante instrucciones en lenguaje natural. Esta aplicación requiere reinterpretar cada principio en el contexto específico de IA: el "mínimo privilegio" incluye limitar el contexto del modelo; la "defense in depth" incluye capas semánticas que operan sobre el significado del texto, no solo sobre su estructura; el "threat modeling continuo" debe actualizarse ante cambios en el comportamiento del modelo que no tienen equivalente en sistemas de software tradicional.

El módulo recorrió este territorio de amenaza a control: desde la taxonomía de amenazas del Capítulo 1 hasta los principios arquitectónicos de este capítulo, pasando por los ataques específicos (Caps. 2-3), la metodología de red teaming (Cap. 4), la seguridad de sistemas RAG y agénticos (Cap. 5), la privacidad de datos y entrenamiento (Cap. 6), el hardening del endpoint (Cap. 7), la trazabilidad y auditoría (Cap. 8), y el compliance regulatorio (Cap. 9). La progresión fue intencional: primero entender el problema con profundidad suficiente para apreciarlo, luego dominar los controles que lo mitigan capa por capa, y finalmente integrar esos controles en principios arquitectónicos que los hacen coherentes como sistema. El AI Engineer que ha recorrido este módulo no solo conoce un inventario de controles de seguridad; tiene el modelo mental para diseñar sistemas de IA que resistan adversarios reales.

Conceptos clave del capítulo

- **Security by design:** las decisiones de trust zones, sandboxing, separación de privilegios y cifrado se toman antes de la primera línea de código; GDPR Art. 25 y EU AI Act Art. 9 lo formalizan como requisito legal; el costo de remediación post-hoc es órdenes de magnitud mayor.
- **Mínimo privilegio:** el modelo recibe solo el contexto mínimo necesario; el agente solo tiene acceso a las herramientas necesarias para la tarea actual; cada herramienta tiene el IAM role más restrictivo posible; privilege elevation temporal con logging para operaciones excepcionales.
- **Defense in depth:** capa de prompt + capa de API (LlamaGuard, rate limiting, autenticación) + capa de red (WAF HTTP + WAF semántico + TLS) + capa de datos (cifrado + RBAC + logs inmutables); cuatro capas independientes con diferentes superficies de ataque.
- **Threat modeling continuo:** actualización obligatoria ante cada nueva capacidad, herramienta o modelo base; mantenido como código en git con versioning explícito y changelog; validado mediante red teaming enfocado en las nuevas amenazas identificadas.
- **Security testing pipeline:** SAST (Semgrep/Bandit) + DAST (ZAP/Burp) + fuzzing de prompts (Garak) + evaluación periódica de clasificadores + red teaming automatizado en

CI/CD; la automatización garantiza cobertura consistente que el proceso manual no puede sostener.

Idea central

Un sistema de IA con diseño seguro por construcción —mínimo privilegio aplicado en cada capa, defensa en profundidad con capas independientes, threat modeling que evoluciona con el sistema, y testing automatizado que verifica los controles— no es uno que nunca es atacado: es uno que cuando es atacado, contiene el daño, lo detecta rápidamente, lo mitiga con las capas de control que el atacante no pudo eludir, y sale más resistente del incidente porque aprende de él de forma sistemática.

"The question is not whether you will be attacked, but whether you will notice and respond appropriately when you are." — Adam Shostack, experto en threat modeling y autor de "Threat Modeling: Designing for Security" (Microsoft Press), sobre el paradigma de resiliencia que debe guiar el diseño de sistemas seguros en la era moderna. Un sistema de IA que no detecta que está siendo atacado no es un sistema seguro, aunque nunca haya tenido un incidente visible: es un sistema que no ha tenido la suerte de encontrar al adversario que lo busca.